

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
ФАКУЛЬТЕТ ПРОГРАММНОЙ ИНЖЕНЕРИИ И КОМПЬЮТЕРНОЙ ТЕХНИКИ
НАПРАВЛЕНИЕ ПРОГРАММНАЯ ИНЖЕНЕРИЯ
ОБРАЗОВАТЕЛЬНАЯ ПРОГРАММА СИСТЕМНОЕ И ПРИКЛАДНОЕ
ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ

ОТЧЕТ ПО РЕШЕНИЮ ЗАДАЧ БЛОКА №2
курса «Алгоритмы и структуры данных»

Выполнил студент:

Исаева А.А.

группа: Р3209

Преподаватель:

Косяков М. С.,

Смирнов В. И.

Санкт-Петербург, 2026 г.

Содержание

1. Задача 5	3
1. Листинг кода	3
2. Описание решения	4
3. Обоснование	4
4. Сложность	4
2. Задача 6	5
1. Листинг кода	5
2. Описание решения	5
3. Обоснование	5
4. Сложность	6
3. Задача 7	6
1. Листинг кода	6
2. Описание решения	8
3. Обоснование	8
4. Сложность	8
4. Задача 8	9
1. Листинг кода	9
2. Описание решения	9
3. Обоснование	10
4. Сложность	10
5. Вывод	10

1. Задача 5

1. 1. Листинг кода

```
1 #include <algorithm>
2 #include <iostream>
3 #include <vector>
4
5 using namespace std;
6
7 int n, k;
8 vector<int> stalls;
9
10 bool check(int dist) {
11     int count = 1;
12     int last_pos = stalls[0];
13
14     for (int i = 1; i < n; ++i) {
15         if (stalls[i] - last_pos >= dist) {
16             count++;
17             last_pos = stalls[i];
18         }
19     }
20
21     return count >= k;
22 }
23
24 int main() {
25     if (!(cin >> n >> k))
26         return 0;
27
28     stalls.resize(n);
29     for (int i = 0; i < n; ++i) {
30         cin >> stalls[i];
31     }
32
33     int l = 0;
34     int r = stalls[n - 1] - stalls[0];
35     int ans = 0;
36
37     while (l <= r) {
38         int mid = l + (r - l) / 2;
39         if (check(mid)) {
```

```

40     ans = mid;
41     l = mid + 1;
42 } else {
43     r = mid - 1;
44 }
45 }
46
47 cout << ans << endl;
48
49 return 0;
50 }

```

Листинг 1: Листинг кода задачи №5

1. 2. Описание решения

Для каждого кандидата на минимальное расстояние проверяется, можно ли расставить K коров так, чтобы между любыми двумя соседними коровами было не менее d . Коровы расставляются последовательно, начиная с первого стойла, каждая следующая корова помещается в первое стойло, которое отстоит от предыдущего не менее чем на d .

1. 3. Обоснование

- Функция `check` монотонна: если можно расставить коров с расстоянием d , то можно и с расстоянием $d' < d$. Это обеспечивает корректность бинарного поиска.
- Размещение коровы как можно раньше не ухудшает возможность разместить последующих коров.
- Диапазон поиска охватывает все возможные значения ответа, поскольку минимальное расстояние не может превышать разницу координат крайних стойл.

1. 4. Сложность

Время: $O(n \log n)$ — бинарный поиск по $O(\log(\text{max_dist}))$ итерациям, каждая из которых выполняет проход за $O(n)$.

Память: $O(n)$ — хранение координат стойл.

2. Задача 6

2. 1. Листинг кода

```
1 #include <algorithm>
2 #include <iostream>
3 #include <string>
4 #include <vector>
5
6 using namespace std;
7
8 bool compareFragments(const string& a, const string&
   b) {
9     return a + b > b + a;
10 }
11
12 int main() {
13     vector<string> fragments;
14     string s;
15
16     while (cin >> s) {
17         fragments.push_back(s);
18     }
19
20     sort(fragments.begin(), fragments.end(),
        compareFragments);
21
22     for (const string& fragment : fragments) {
23         cout << fragment;
24     }
```

Листинг 2: Листинг кода задачи №6

2. 2. Описание решения

Задача сводится к нахождению такого соединения фрагментов, при котором получается наибольшее число. Для сравнения двух фрагментов a и b используется компаратор: a предпочтительнее b , если $\overline{ab} > \overline{ba}$. После сортировки фрагменты выводятся подряд.

2. 3. Обоснование

- Отношение порядка, задаваемое $ab > ba$, транзитивно и задаёт полный порядок на множестве фрагментов, что делает сортировку корректной.
- Если $ab > ba$, то в оптимальном порядке a должен стоять перед b : перестановка только ухудшит результат.

2. 4. Сложность

Время: $O(n \cdot L \cdot \log n)$, где n — количество фрагментов, L — максимальная длина фрагмента

Память: $O(n \cdot L)$ — хранение всех фрагментов.

3. Задача 7

3. 1. Листинг кода

```

1 #include <algorithm>
2 #include <iostream>
3 #include <string>
4 #include <vector>
5
6 using namespace std;
7
8 struct Letter {
9     char symbol;
10    long long value;
11 };
12
13 bool compare(const Letter& a, const Letter& b) {
14     return a.value > b.value;
15 }
16
17 int main() {
18     string s;
19     if (!(cin >> s))
20         return 0;
21
22     int n = s.length();
23     vector<long long> weights(26);
24     for (int i = 0; i < 26; ++i) {
25         cin >> weights[i];
26     }
27

```

```

28 vector<int> counts(26, 0);
29 for (char c : s) {
30     counts[c - 'a']++;
31 }
32
33 vector<Letter> pairs;
34 for (int i = 0; i < 26; ++i) {
35     if (counts[i] >= 2) {
36         pairs.push_back({(char)('a' + i), weights[i]});
37     }
38 }
39
40 sort(pairs.begin(), pairs.end(), compare);
41
42 string res(n, ' ');
43 vector<bool> fixed(n, false);
44 int L = 0;
45 int R = n - 1;
46
47 for (const auto& item : pairs) {
48     if (L >= R)
49         break;
50
51     res[L] = item.symbol;
52     res[R] = item.symbol;
53     fixed[L] = true;
54     fixed[R] = true;
55
56     counts[item.symbol - 'a'] -= 2;
57
58     L++;
59     R--;
60 }
61
62 int idx = 0;
63 for (int i = 0; i < n; ++i) {
64     if (!fixed[i]) {
65         while (idx < 26 && counts[idx] == 0) {
66             idx++;
67         }
68         if (idx < 26) {
69             res[i] = (char)('a' + idx);
70             counts[idx]--;

```

```

71     }
72   }
73 }
74
75 cout << res << endl;
76
77 return 0;
78 }

```

Листинг 3: Листинг кода задачи №7

3. 2. Описание решения

Чтобы максимизировать вес строки, буквы с большим весом должны располагаться как можно дальше друг от друга — то есть одна на самой левой, другая на самой правой доступной позиции. Алгоритм жадно заполняет позиции с двух концов строки буквами в порядке убывания веса.

- Подсчитываются частоты каждой буквы в строке. Формируется список пар для букв, встречающихся не менее двух раз, и сортируется по убыванию веса.
- Два указателя L и R движутся от краёв к центру. Для каждой буквы из отсортированного списка (пока $L < R$) она ставится на позиции L и R , указатели сдвигаются внутрь, счётчик буквы уменьшается на 2.
- Оставшиеся буквы расставляются в незафиксированные позиции в произвольном порядке.

3. 3. Обоснование

- Вклад буквы в вес определяется расстоянием между её крайними вхождениями. Чтобы максимизировать этот вклад, крайние вхождения должны быть на позициях 0 и $n - 1$.
- Буква с наибольшим весом должна занимать позиции 0 и $n - 1$, следующая — позиции 1 и $n - 2$, и т. д. Жадный выбор по убыванию веса обеспечивает оптимальность.
- Буквы, встречающиеся лишь один раз, не дают вклада в вес независимо от позиции, поэтому их расположение не влияет на результат.

3. 4. Сложность

Время: $O(n + 26 \log 26) = O(n)$ — подсчёт частот за $O(n)$, сортировка 26 букв за $O(1)$, заполнение строки за $O(n)$.

Память: $O(n)$ — результирующая строка и вспомогательные массивы размера $O(26) = O(1)$.

4. Задача 8

4. 1. Листинг кода

```
1 #include <algorithm>
2 #include <iostream>
3 #include <vector>
4
5 using namespace std;
6
7 int main() {
8     int quantity, step;
9     if (!(cin >> quantity >> step))
10         return 0;
11
12     vector<int> prices(quantity);
13     long long total = 0;
14
15     for (int i = 0; i < quantity; i++) {
16         cin >> prices[i];
17         total += prices[i];
18     }
19
20     sort(prices.begin(), prices.end(), greater<int>());
21
22     long long discount = 0;
23     for (int i = step - 1; i < quantity; i += step) {
24         discount += prices[i];
25     }
26
27     cout << total - discount << endl;
28
29     return 0;
30 }
```

Листинг 4: Листинг кода задачи №8

4. 2. Описание решения

- Считываются все цены, вычисляется их суммарная стоимость.
- Цены сортируются по убыванию. В отсортированном массиве бесплатными становятся элементы с индексами $k - 1, 2k - 1, 3k - 1, \dots$ - самые дорогие из каждой группы k товаров.
- Суммируется скидка по этим позициям

4. 3. Обоснование

- Разбиение на несколько чеков не даёт выгоды: при одном чеке из n товаров бесплатно достаётся $\lfloor n/k \rfloor$ штук, тогда как разбиение на группы даёт суммарно не больше бесплатных товаров
- При фиксированном числе бесплатных товаров скидка максимальна, если бесплатными оказываются наиболее дорогие из допустимых. Сортировка по убыванию обеспечивает именно это.
- Жадный выбор оптимален: каждый k -й элемент в отсортированном по убыванию массиве — наибольший из «хвостов» групп по k товаров.

4. 4. Сложность

Время: $O(n \log n)$ — сортировка массива цен.

Память: $O(n)$ — хранение массива цен.

5. Вывод

В ходе решения задач были применены фундаментальные алгоритмические подходы и структуры данных, рассмотренные в курсе. Задача 5 продемонстрировала применение бинарного поиска по ответу в сочетании с жадной проверкой. Задача 6 показала, что задача об оптимальном соединении строк сводится к сортировке с нестандартным компаратором. В задаче 7 жадное заполнение строки с двух концов в порядке убывания весов букв позволяет получить максимальный вес. Задача 8 решается жадным алгоритмом: сортировка цен по убыванию и размещение всех товаров в один чек гарантируют максимальную скидку.